JIMMA CITY ADMINISTRATION

Jimma City Smart Asset Management System

Security Documentation

Security controls, threat model, test evidence and residual risks

This document is the security description of the Jimma City Smart Asset Management System for the INSA web application and API assessment. It states what the system protects, against which threats, with which controls, how those controls were tested, what remains to be done at deployment, and which risks are known and accepted.

Document number	JCSAMS-SEC-005
Version	1.0 (submission issue)
Date of issue	25 October 2026
System owner	Jimma City Administration, Ethiopia
Developed by	Banmen Technologies
Prepared by	Bealem Sintayehu, Project Lead, Banmen Technologies
Submitted to	Information Network Security Administration (INSA), through the Hibir platform
Assessment requested	Web application and API security assessment (penetration test)
Classification	RESTRICTED

Document control

Revision history

Version	Date	Author	Description of change
0.5	16 October 2026	Bealem Sintayehu	Controls mapped to the OWASP Top 10.
0.8	24 October 2026	Bealem Sintayehu	Account lockout, idle timeout, server-side sign-out, upload content checks, temporary password expiry, secret-free logging added and tested.
1.0	25 October 2026	Bealem Sintayehu	Issued for the INSA assessment with threat model, permission matrix and test evidence.

Review and approval

This document is valid for submission once the people below have reviewed and signed it. Signatures are collected on the printed copy kept in the project file at the Jimma City Administration.

Role	Name	Position	Signature	Date
Prepared by	Bealem Sintayehu	Project Lead, Banmen Technologies		
Reviewed by		Head, ICT Directorate, Jimma City Administration		
Approved by		Office of the Mayor, Jimma City Administration		

Distribution

Copy	Recipient	Form
1	Information Network Security Administration (INSA), assessment team	Electronic, through the Hibir platform
2	Office of the Mayor, Jimma City Administration	Printed and electronic
3	ICT Directorate, Jimma City Administration	Printed and electronic
4	Project file	Printed, signed original

Related documents in this submission

Number	Title
JCSAMS-LTR-000	Covering letter to INSA
JCSAMS-SRS-001	Software Requirements Specification
JCSAMS-SDD-002	System and Architecture Design Document
JCSAMS-TDD-003	Technical Design and Specification
JCSAMS-UG-004	User Guide
JCSAMS-SEC-005	Security Documentation

Contents

1 Introduction . . . 4

1.1 Purpose . . . 4

1.2 Scope of the assessment . . . 4

1.3 Test environment and accounts . . . 4

2 System summary for the assessor . . . 4

3 What is protected . . . 5

3.1 Information assets . . . 5

3.2 Security objectives . . . 6

4 Threat model . . . 6

5 Security controls . . . 7

5.1 Authentication . . . 7

5.2 Authorisation . . . 8

5.3 Input validation and output encoding . . . 9

5.4 File uploads and downloads . . . 9

5.5 Transport and headers . . . 9

5.6 Cryptography . . . 10

5.7 Logging and audit . . . 10

5.8 AI assistant . . . 10

5.9 Error handling . . . 11

5.10 Dependencies . . . 11

6 Mapping to OWASP lists . . . 11

6.1 OWASP Top 10 (2021) . . . 11

6.2 OWASP API Security Top 10 (2023) . . . 11

7 Secure deployment checklist . . . 12

8 Security testing performed . . . 13

9 Known limitations and residual risks . . . 14

10 Incident response and vulnerability reporting . . . 14

10.1 Incident response . . . 14

10.2 Reporting a vulnerability . . . 15

1 Introduction

1.1 Purpose

Jimma City Administration asks INSA to assess the security of the Jimma City Smart Asset Management System before it is put into production. The system will hold the City's register of public assets, their values and income, and the accounts of about 194 City staff. A breach could allow someone to hide or invent assets, alter values, read confidential financial information or impersonate officials in the approval chain. This document gives the assessment team the information needed to test the system efficiently and to judge the findings in context.

1.2 Scope of the assessment

In scope	Out of scope
The web application (all pages, all eight roles) The REST API under /api, including the public card, the integration API and the AI assistant Authentication, session handling and authorisation File upload and download The QR label and public card mechanism Security headers and TLS configuration of the test server	Denial-of-service and load testing Physical security of the City data centre Social engineering of City staff The SMTP server and SMS gateway themselves Third-party map tile services

1.3 Test environment and accounts

A dedicated test installation, separate from any production data, is provided for the assessment. It contains realistic but fictitious assets and one account for each of the eight roles, plus a second Sector Administrator in another Kebele for testing access between areas. The address, the accounts and their passwords are handed to the assessment team separately, not in this document, as agreed in the covering letter. Accounts are created with temporary passwords that must be changed at first sign-in. On request we can switch the rate limits higher for automated scanning, and we will report the exact settings used.

2 System summary for the assessor

A full description is in the System and Architecture Design Document (JCSAMS-SDD-002). The points that matter most for testing are:

- Single-page React application and a JSON REST API served by one Node.js (Express 5) process behind an nginx TLS proxy; MySQL 8 on the same host.
- Session tokens are JWTs (HS256) sent in the Authorization header; there are no session cookies, so classic CSRF does not apply. The token is held in sessionStorage.
- Eight roles with permissions stored in the database; every user also has an area (City, Kifle Ketema, Kebele, or sector in a Kebele). Both are checked on the server for every request.
- Unauthenticated surface: sign-in, translations, enumerations, health, and the public asset card (`/api/public/assets/:token`, also reached through `/a/:token`).
- Machine surface: the read-only integration API (`/api/integration/v1`) authenticated with an X-API-Key header.
- File uploads: one file per request, 10 MB, content checked by its first bytes; stored under random names; downloaded only through an authenticated route.

- AI assistant: `POST /api/assistant/chat` (signed in). The server sends the question to the configured AI service with four read-only lookup functions that run as the user. Prompt-injection attempts are in scope; the design goal is that even a fully manipulated model cannot read outside the user's area or change anything.

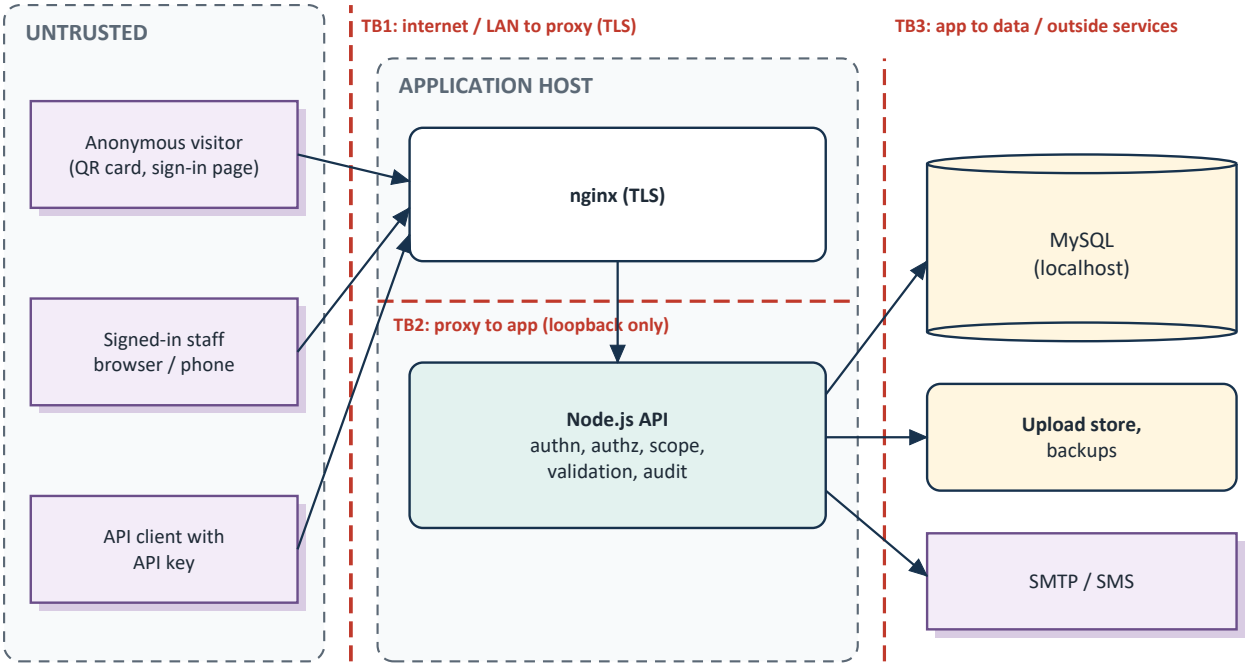


Figure 1: Trust boundaries

3 What is protected

3.1 Information assets

Asset	Sensitivity	Main threat	Where it is protected
Asset register and its history	High (integrity)	Fictitious or hidden assets, falsified condition or location	Registration chain with separation of duties; audit log; area scope
Values, income, purchase and contract documents	High (confidentiality, integrity)	Disclosure; manipulation of values	financial.view / financial.edit permissions; Financial Officers only record
Staff accounts and personal data	High	Account takeover; enumeration; disclosure of phone numbers	Password rules, lockout, sessions, account visibility by hierarchy
Approval decisions	High (integrity, n on-repudiation)	Approving one's own request; impersonation	Separation of duties; per-user accounts; audit log
Monthly reports	Medium	Altering a sent report	Frozen snapshot; only sender and receiver; accepted reports locked
Audit log	High (integrity)	Deletion or alteration to hide actions	Insert-only by design; readable only by the Super Administrator and Internal Auditor
Secrets (JWT secret, DB password, API keys)	Critical	Theft leading to full compromise	Server configuration file with restricted permissions; API keys stored hashed

3.2 Security objectives

1. Only a signed-in person with the right role and area can see or change a record (confidentiality and integrity).
2. No single person can bring an asset into the register, or approve a change to it, alone (integrity through separation of duties).
3. Every change can be traced to a named person and a time (accountability).
4. Financial information is visible only to roles that need it (confidentiality).
5. The register can be restored after a failure with at most one day of data lost (availability).

4 Threat model

We used STRIDE on each trust boundary and data flow of the design (flow identifiers F1 to F14 are defined in JCSAMS-SDD-002, Table 6). Table 3 lists the threats we consider realistic for this system, the control that addresses each one, and the risk that remains after the control, rated Low, Medium or High. STRIDE letters: S spoofing, T tampering, R repudiation, I information disclosure, D denial of service, E elevation of privilege.

#	STRIDE	Threat	Control	Residual
T1	S	Password guessing or credential stuffing against sign-in (F1)	Lock after 5 failures for 15 min per account; 20 attempts/min per address; strong password rule; bcrypt cost 12	Low
T2	S	Username enumeration through messages or timing	Same message and same bcrypt time for unknown users; unknown users throttled the same way	Low
T3	S	Stolen session token reused (F2)	Token only in sessionStorage; 30-min idle end; 8-hour maximum; server-side sign-out; TLS; strict CSP limits script injection	Low
T4	S	Shared or never-changed initial passwords	Random temporary passwords, forced change, 7-day expiry; credential file to be deleted after hand-out	Low
T5	T	Changing a record outside one's area by altering an ID (IDOR) (F3 to F6)	Every query scoped by area; guardAsset on every asset route; 404 outside area	Low
T6	T	Approving one's own registration or request	Separation of duties enforced on the server at each step	Low
T7	T	SQL injection through any field	Knex bound parameters; schema validation; LIKE wildcards escaped	Low
T8	T	Uploading a script or HTML disguised as a document (F3, F9)	Allow-list of types; content checked by signature; random names; served as attachment with nosniff; no HTML or SVG	Low
T9	T	Altering monthly figures after sending (F13)	Snapshot frozen at send; accepted report locked; new version on return	Low
T10	R	Denying having made a change or decision	Per-person accounts; insert-only audit log with old and new values	Low
T11	I	Financial data seen by unauthorised roles	financial.view checked when building each response; money documents hidden	Low
T12	I	Data leakage through the public card or integration API (F10, F11)	Fixed list of safe fields; no money, names, GPS or serials; tokens random 16 characters	Low
T13	I	Error messages revealing internals	Generic 500 message; details only in the server log	Low

#	STRIDE	Threat	Control	Residual
T14	I	Secrets in logs (temporary passwords in SMS text)	Logs record only recipient and length of messages	Low
T15	I	Data in browser or proxy caches	Cache-Control: no-store on all API answers; service worker never caches /api	Low
T16	D	Request flooding of the API	Rate limits per address; 1 MB body limit; 10 MB upload limit; paging limits	Medium (single server)
T17	E	Lower role gaining a higher role's functions	Permissions checked on the server; account management only one level down; role changes end sessions	Low
T18	E	Script injection (XSS) to act as another user	React escapes output; strict CSP without inline scripts; map pop-ups escaped	Low
T19	E	Compromise of the server configuration (JWT secret)	Secret required at start-up; file permissions 0600; deployment checklist	Medium (operational)
T20	E	Prompt injection (in the question or in record text) to make the AI reveal other areas' data or act	The AI has only four read-only lookups, each run with the user's own scope and money mask; no write function exists; instructions only from the server	Low
T21	I	Questions and lookup results kept by the AI provider (F15)	Off unless configured; local model possible; City decision before real data (SRS OI-8)	Medium (free cloud tiers)
T22	D	Abuse of the assistant to exhaust the free AI quota	10 questions/min per person; 12 turns of 2000 characters; 45 s time limit; at most 4 rounds per question	Low
T23	T	Staff acting on a wrong AI answer	Answers marked as possibly wrong; every figure comes from a lookup; decisions are still taken on the record pages	Low

Table 3: Threats, controls and residual risk

5 Security controls

5.1 Authentication

Control	Implementation
Password storage	bcrypt, cost 12. Hashes are never returned by the API or written to logs.
Password rule	8 to 128 characters, at least one upper-case letter, lower-case letter, digit and symbol; checked in the form and on the server.
Account lockout	5 wrong passwords in a row lock the account for 15 minutes from any address. Lock and failed attempts are audited.
Rate limit	20 sign-in attempts per minute per address; 600 API requests per minute per address.
Enumeration resistance	Identical message and timing for unknown users and wrong passwords.
Temporary passwords	Random 14 characters meeting the rule, no look-alike characters; must be changed at first sign-in; expire after 7 days.
Session token	JWT HS256, signed with a secret of at least 16 characters (48 bytes recommended); contains only user id, token version and expiry.
Session end	30 minutes idle (server and screen), 8 hours maximum; sign-out, password change, role change, deactivation and "end sessions" invalidate all tokens through the token version.

Control	Implementation
Token storage	sessionStorage (cleared when the browser closes); never in URLs or cookies.
Not implemented	Two-factor authentication and single sign-on (planned; see section 9).

5.2 Authorisation

Authorisation has two independent parts that must both allow a request: the **permission** of the role, and the **area** of the user. Table 4 shows the default permissions of each role. The Super Administrator can change them, but cannot remove its own administration rights. Any change takes effect on the next request.

Permission	Meaning	SA	MY	KK	KB	SC	IA	FO	IN
`asset.create`	Register assets					**Yes**			
`asset.edit`	Edit asset records				**Yes**	**Yes**			
`asset.review`	Verify new assets on site				**Yes**				
`asset.approve`	Approve and activate assets			**Yes**					
`request.create`	Raise requests				**Yes**	**Yes**			
`request.review`	Review requests (step 1)				**Yes**				
`request.decide`	Decide requests (final)			**Yes**					
`maintenance.record`	Record maintenance					**Yes**			
`inspection.record`	Record inspections								**Yes**
`inspection.oversee`	Follow Inspectors	**Yes**		**Yes**					
`verification.record`	Record verification results				**Yes**				**Yes**
`verification.manage`	Run campaigns, approve corrections			**Yes**	**Yes**				
`reservation.request`	Request reservations				**Yes**	**Yes**			
`reservation.decide`	Decide reservations				**Yes**	**Yes**			
`financial.view`	See money		**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	
`financial.edit`	Record valuations and income							**Yes**	
`asset.export`	Export lists, find duplicates	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**
`analytics.view`	See analytics		**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	
`import.run`	Bulk import								
`audit.view`	Read the audit log	**Yes**					**Yes**		
`master.manage`	Manage master data	**Yes**							
`user.manage`	Manage Kifle Ketema admins and specialists	**Yes**							
`user.manage_kebele_admins`	Manage Kebele admins			**Yes**					
`user.manage_sector_admins`	Manage Sector admins				**Yes**				
`system.manage`	Workflows, permissions, settings, backups, API keys	**Yes**							

Table 4: Default role permissions. SA Super Administrator, MY Mayor / Executive, KK Kifle Ketema Administrator, KB Kebele Administrator, SC Sector Administrator, IA Internal Auditor, FO Financial Officer, IN Inspector.

- **Area scope.** Every query for assets and their records is limited to the user's area. A record outside the area answers 404 so that its existence is not revealed.
- **Account visibility.** Each level sees and manages only the accounts of the level directly below it in its own area; the Internal Auditor reads all accounts without changing them.
- **Separation of duties.** The registrar cannot verify, the verifier cannot approve, a requester cannot review or decide, a reviewer cannot give the final decision, and the person who recorded a verification finding cannot approve its correction.
- **Function-level checks.** The menu hides what a role cannot use, but every API route checks the permission itself.
- **Mass assignment.** Input is parsed into explicit schemas; fields such as role, area, approval state, created_by or asset_code cannot be set through ordinary update requests.

5.3 Input validation and output encoding

- Every request body and query string is validated by a zod schema before use: types, lengths, allowed values, dates, money, names, phone numbers, coordinates inside the City. Unknown fields are dropped.
- All SQL is built with Knex and bound parameters. Free-text search escapes % and _ before it is used in LIKE.
- The React interface renders all text escaped; the application does not insert HTML built from user input. Map pop-ups escape their text.
- CSV exports prefix cells beginning with =, +, - or @ so that spreadsheet programs do not execute them.
- Malformed JSON and unexpected types return 400 with a clear message; they never crash the process.

5.4 File uploads and downloads

Aspect	Control
Allowed types	PDF, JPG, PNG, WebP, DOC, DOCX, XLS, XLSX. No HTML, SVG, scripts or archives.
Content check	The first bytes of the saved file must match the signature of the declared type (for example %PDF for PDF); otherwise the file is deleted and refused.
Size and count	10 MB, one file per request; request bodies without files 1 MB.
Storage	Random file names in the upload folder, outside the web root; original names kept only in the database.
Download	Only through an authenticated route that checks the asset's area and, for purchase, invoice and contract documents, the financial permission; sent as an attachment with nosniff; every download audited.

5.5 Transport and headers

Production runs only over HTTPS (TLS 1.2 or 1.3) at the nginx proxy; HTTP only redirects. When the public address is HTTPS the application also sends HSTS. The full list of headers and their values is in JCSAMS-TDD-003, section 4.2. Summary: a strict Content Security Policy (scripts only from the site itself, no inline scripts, object-src none, frame-ancestors self), nosniff, X-Frame-Options SAMEORIGIN, a restrictive Permissions-Policy, Referrer-Policy strict-origin-when-cross-origin, cross-origin isolation headers, and Cache-Control no-store for every API answer. X-Powered-By is removed. CORS allows only the application's own origin.

5.6 Cryptography

Use	Algorithm	Notes
Passwords	bcrypt, cost 12	Per-password salt included in the hash.
Session tokens	HMAC-SHA256 (JWT HS256)	Server-side secret; rotation by changing JWT_SECRET ends all sessions.
API keys	SHA-256 of a random key	Key shown once at creation; only the hash is stored.
Public label tokens	12 random bytes (crypto.randomBytes), base64url	16 characters; not derived from the Asset ID.
Temporary passwords	crypto.randomInt	Cryptographically secure random choice of characters.
Transport	TLS 1.2 / 1.3 at nginx	Certificate for the City domain.
Data at rest	Disk encryption (LUKS) at the server level	The application itself does not encrypt database fields.

5.7 Logging and audit

- The audit log records: sign-in, failed sign-in, account lock, sign-out, password change; every create, change and status change of assets, accounts, permissions, master data and settings; every approval decision; every document upload, download and removal; every export; backups and API key changes. Each entry has the user, the time, the action, the record and the old and new values.
- The audit log is insert-only: the application has no update or delete operation for it, and only the Super Administrator and the Internal Auditor can read it.
- The server log (standard output, collected by systemd) records errors with their technical detail. It never contains passwords, temporary passwords, tokens or API keys; for e-mail and SMS it records only the recipient and the message length.

5.8 AI assistant

- Switched off unless the AI service is configured (AI_BASE_URL, AI_MODEL and, for a cloud service, AI_API_KEY). Any OpenAI-compatible service works, including Ollama on the City's own server.
- **Read-only by construction.** The model can call only search_assets, count_assets, get_asset and my_work. They run through the same area-scoped queries as the screens, money is removed for roles without financial.view, and unknown tools are refused.
- **Server-owned instructions.** The browser may send only user and assistant turns (at most 12, 2000 characters each); a system message from the browser is refused with 400. Record texts are passed as data, and the instructions tell the model to ignore instructions inside data.
- **Limits.** 10 questions a minute per person, 45-second time limit per call, at most 4 tool rounds, tool results cut at 6000 characters.
- **Logging.** The audit log notes each use and the lookups made, not the question; the provider's error texts and the API key are never logged.
- **Output.** Answers are reduced to plain text on the server and shown as text (never as HTML), so a manipulated answer cannot inject script.
- **Availability.** When the AI service is overloaded the server retries once; when the free quota is used up the user is asked to wait. The rest of the system does not depend on the assistant.

5.9 Error handling

Errors are answered in one JSON shape with an HTTP status code (see JCSAMS-TDD-003, section 7). Database errors such as duplicate keys are translated into plain messages. Any unexpected error returns only "Internal server error."; stack traces, SQL statements, file paths and library versions are never sent to the client.

5.10 Dependencies

All package versions are locked in `package-lock.json`. On 25 October 2026, after updating zod and nodemailer, `npm audit` reported 0 known vulnerabilities in the backend and in the frontend. The audit is repeated before every release.

6 Mapping to OWASP lists

6.1 OWASP Top 10 (2021)

Risk	How the system addresses it	Section
A01 Broken Access Control	Server-side permission and area checks on every route; 404 outside area; hierarchy for accounts; separation of duties; CORS own origin.	5.2
A02 Cryptographic Failures	bcrypt; HS256 tokens with a strong secret; API keys hashed; HTTPS with HSTS; no-store caching.	5.6
A03 Injection	Bound parameters; schema validation; LIKE escaping; CSV formula neutralisation; escaped output and strict CSP.	5.3
A04 Insecure Design	Threat model; approval chain with separation of duties; rate limits; body and upload limits.	4, 5.2
A05 Security Misconfiguration	Security headers; no stack traces; no X-Powered-By; deployment checklist; no directory listing.	5.5, 7
A06 Vulnerable and Outdated Components	Locked versions; npm audit with 0 known vulnerabilities; update procedure.	5.10
A07 Identification and Authentication Failures	Password rule; lockout; enumeration resistance; idle and absolute timeouts; server-side sign-out; temporary password expiry.	5.1
A08 Software and Data Integrity Failures	Upload content checks; locked dependencies; frozen report snapshots; insert-only audit log.	5.4
A09 Security Logging and Monitoring Failures	Complete audit log; secret-free server log; backup health warnings.	5.7
A10 Server-Side Request Forgery	The server never fetches a user-supplied URL; outbound only to configured SMTP and SMS.	2

6.2 OWASP API Security Top 10 (2023)

Risk	How the system addresses it
API1 Broken Object Level Authorization	Every object route loads the object through the area-scoped query; others answer 404.
API2 Broken Authentication	As A07 above; API keys for machine access are hashed and can be disabled at once.
API3 Broken Object Property Level Authorization	Responses built per permission (money removed without financial.view); explicit input schemas prevent mass assignment.

Risk	How the system addresses it
API4 Unrestricted Resource Consumption	Rate limits; 1 MB body; 10 MB upload; page size at most 100; one file per request.
API5 Broken Function Level Authorization	Each route checks its permission on the server; administrative routes require system.manage.
API6 Unrestricted Access to Sensitive Business Flows	Approval flows need different people at each step; reservations refuse overlaps.
API7 Server Side Request Forgery	No user-supplied URLs are fetched.
API8 Security Misconfiguration	Headers, CORS, error handling and deployment checklist as above.
API9 Improper Inventory Management	All endpoints listed in JCSAMS-TDD-003; one API version for integration (v1) with an OpenAPI description; no test or debug endpoints in the build.
API10 Unsafe Consumption of APIs	External APIs are SMTP, the SMS gateway and the AI service. Delivery answers only update a status; AI answers are shown as plain text and can only trigger the read-only, user-scoped lookups.

7 Secure deployment checklist

Some findings are only prevented on the server. Table 7 is the checklist the City's ICT Directorate completes before go-live; the test server for the assessment is set up the same way.

#	Item	How	Done
1	HTTPS only, TLS 1.2/1.3, valid certificate	nginx with the City certificate; HTTP redirects; PUBLIC_URL=https://... so HSTS is sent	
2	Strong secrets	JWT_SECRET of 48 random bytes; long random database password; .env mode 0600; never committed	
3	Database least privilege	Application user with SELECT, INSERT, UPDATE, DELETE only outside upgrades; MySQL bound to 127.0.0.1	
4	Firewall	Only 443 and 80 open to users; SSH only from the administration network; 4000 and 3306 closed	
5	Service account	Node.js runs as an unprivileged user under systemd with NODE_ENV=production and restart on failure	
6	Upload folder	Outside the web root, not executable, owned by the service account, on an encrypted disk	
7	Backups	Daily; copied off the server to encrypted storage; restore tested every quarter	
8	Updates	OS security updates monthly; npm audit before every release	
9	Logs	Server and nginx logs kept at least 6 months; audit log kept in the database	
10	No default or test accounts	Only real accounts provisioned; credential CSV deleted after hand-out; all temporary passwords replaced	
11	No development servers	The Vite development server (port 5173) is never exposed; only the built files are served	
12	Separation of duties on	SEPARATION_OF_DUTIES=true	

Table 7: Deployment checklist

Example nginx server block:

```
server {
    listen 443 ssl http2;
    server_name assets.jimma.gov.et;
    ssl_certificate     /etc/ssl/jimma/fullchain.pem;
    ssl_certificate_key /etc/ssl/jimma/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;
    server_tokens off;
    client_max_body_size 12m;
    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto https;
    }
}
server { listen 80; server_name assets.jimma.gov.et; return 301 https://$host$request_uri; }
```

8 Security testing performed

The automated test suite (265 tests, all passing on 25 October 2026) contains the following security tests among others. They call the real API over HTTP against a real MySQL test database.

Area	What the tests prove
Authentication	Wrong passwords lock the account after 5 attempts and the lock message is shown; unknown and known users get the same answer; the password rule is enforced; sign-out ends the token; an idle session ends; an expired temporary password is refused; a temporary password blocks all other functions.
Sessions	A password change, role change, deactivation or "end sessions" makes older tokens fail.
Access by role	Each role is refused the functions it lacks (for example the Super Administrator cannot register or approve assets; only Inspectors record inspections; only Financial Officers record money).
Access by area (IDOR)	A user asking for an asset, document, user or report outside their area gets 404, including by guessing IDs.
Account hierarchy	Each level can create and see only the accounts of the level below in its area; the Internal Auditor sees all and can change none.
Separation of duties	The registrar cannot verify, the verifier cannot approve; a requester cannot decide; the finder of a discrepancy cannot approve its correction.
Injection and malformed input	SQL metacharacters in every searchable field return normal results; malformed JSON, wrong types, missing and unexpected properties return 400 without crashing.
Uploads	A file whose content does not match its type is refused and deleted; oversized files are refused; HTML and SVG are refused.
Financial privacy	Roles without the financial permission receive asset records without money fields and cannot download purchase, invoice or contract documents.
Public card	Only the safe fields are returned; replaced labels answer 410; unknown tokens answer 404.
Headers	Security headers and Cache-Control: no-store are present on API answers.
AI assistant	Off until configured; needs sign-in; refuses a browser-supplied system message and over-long input; lookups never return assets outside the user's area; money reaches the AI only for financial roles; an unknown (for example a delete) tool changes nothing; the audit entry contains no question text.
CSV	Exported cells starting with formula characters are neutralised.

Not yet performed: an automated scan with OWASP ZAP against the HTTPS test server, and an independent penetration test. These are exactly what we ask INSA to carry out.

9 Known limitations and residual risks

We list these openly so the assessment can take them into account. Each has an owner and a planned treatment.

#	Limitation	Risk	Treatment
L1	No two-factor authentication.	Medium: a stolen password is enough to sign in (lockout and idle timeout limit the damage).	Add TOTP for administrators in the next release; until then, strong passwords and lockout.
L2	Rate limits are counted in memory.	Low on one server; ineffective across several servers; reset on restart.	Accepted for a single server; use a shared store (Redis) if the system is ever scaled out.
L3	The application does not encrypt database fields.	Medium if the disk or backups are stolen.	Full-disk encryption on the server and encrypted off-site backups (checklist 6, 7).
L4	Session token is readable by scripts in the page (sessionStorage).	Low given the strict CSP and escaped output.	Accepted; reviewed again if inline scripts or third-party scripts are ever added.
L5	JWT secret rotation ends all sessions.	Low (operational inconvenience only).	Rotate during a maintenance window.
L6	Translations prepared without professional review.	Low (security messages could be misunderstood).	Review by the City's language staff before go-live.
L7	Load testing not done.	Medium for availability at full volume.	Load test before go-live (SRS NFR-PRF-01).
L8	On a free cloud AI tier, the provider may keep questions and lookup results.	Medium (confidentiality of internal data; money only for financial roles).	Use a local model (Ollama) or a paid tier without data use before real data; City decision recorded (SRS OI-8).

10 Incident response and vulnerability reporting

10.1 Incident response

1. **Detect.** Unusual activity is found in the audit log (for example many failed sign-ins or locks), in the server log, or reported by a user.
2. **Contain.** The Super Administrator deactivates affected accounts or ends their sessions; if the server itself may be compromised, the ICT Directorate takes it off the network and changes JWT_SECRET, which ends every session.
3. **Investigate.** The Internal Auditor and the ICT Directorate use the audit log, which records the old and new values of every change, to establish what happened.
4. **Recover.** Wrong data is corrected through the normal functions (so the correction is audited) or, if needed, the last good backup is restored.
5. **Report.** Serious incidents are reported to the Office of the Mayor and, as required by Ethiopian law, to INSA's Ethiopian Computer Emergency Response Team (Ethio-CERT).
6. **Learn.** The cause is fixed, tested and recorded in this document's revision history.

10.2 Reporting a vulnerability

Findings from the INSA assessment and from anyone else should be sent to the system owner, the ICT Directorate of the Jimma City Administration, and to the project lead. We will confirm receipt, fix confirmed findings in order of severity, re-run the test suite, and provide a written response for each finding before re-testing.